```
import torch
import torchvision
import torchvision.transforms as transforms

# Transform (Normalize between -1 and 1)
transform = transforms.Compose([
    transforms.Resize(64),
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# Load CIFAR-10 dataset
dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=True,
    transform=transform,
    download=True
)

dataloader = torch.utils.data.DataLoader(dataset, batch_size=64, shuffle=Tru

# Latent vector dimension
latent_dim = 100
```

```
100%|██████████| 170M/170M [00:04<00:00, 39.8MB/s]
```

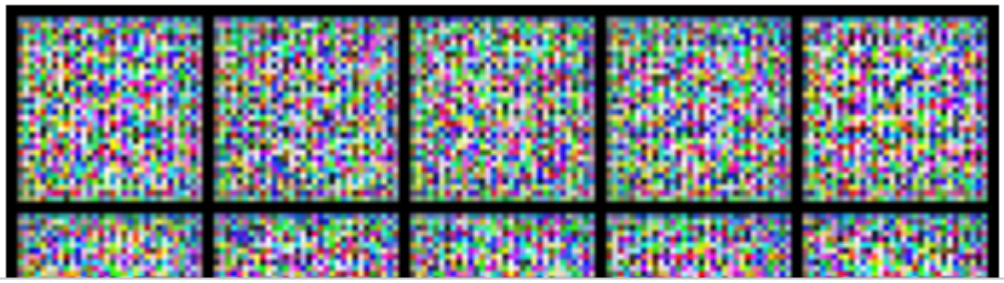
```
import torch.nn as nn

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

class Generator(nn.Module):
    def __init__(self, latent_dim):
        super().__init__()

        self.model = nn.Sequential(
            nn.ConvTranspose2d(latent_dim, 512, 4, 1, 0),
            nn.BatchNorm2d(512),
            nn.ReLU(),

            nn.ConvTranspose2d(512, 256, 4, 2, 1),
            nn.BatchNorm2d(256),
            nn.ReLU(),

            nn.ConvTranspose2d(256, 128, 4, 2, 1),
            nn.BatchNorm2d(128),
            nn.ReLU(),

            nn.ConvTranspose2d(128, 3, 4, 2, 1),
            nn.Tanh()
        )

    def forward(self, x):
```

```
        return self.model(x)

generator = Generator(latent_dim).to(device)

# Freeze generator weights
for param in generator.parameters():
    param.requires_grad = False
```

```
import matplotlib.pyplot as plt
import numpy as np

def generate_images(n):
    noise = torch.randn(n, latent_dim, 1, 1).to(device)
    images = generator(noise).detach().cpu()
    return images
```

```
def show_images(images):
    grid = torchvision.utils.make_grid(images, nrow=5, normalize=True)
    plt.imshow(np.transpose(grid, (1,2,0)))
    plt.axis('off')
    plt.show()
```

```
images = generate_images(10)
show_images(images)
```



```
def interpolate(z1, z2, steps=10):
    result = []
    for alpha in np.linspace(0, 1, ste
        z = (1 - alpha) * z1 + alpha *
        result.append(z)
    return torch.cat(result)

z1 = torch.randn(1, latent_dim, 1, 1).
z2 = torch.randn(1, latent_dim, 1, 1).

z_interp = interpolate(z1, z2)
images = generator(z_interp).detach().

show_images(images)
```

Start coding or generate with AI.

Start coding or generate with AI.